# Polymarket Tweet-Count Bins: Kelly Utility Maximization Trading Spec

Implementation handoff for Claude (CLOB, long YES/NO only, hold-to-settlement)

**Scope**: This document specifies a complete, precise trading algorithm for a Polymarket CLOB event whose outcomes are bins of Elon Musk tweet counts over a fixed window. The strategy trades up to a cutoff time T_stop, then holds all remaining positions to settlement. The sizing and trade/no-trade decisions are driven by exact expected log-utility (Kelly) maximization under hard collateral caps and the constraint that the bot can only hold non-negative YES and NO positions (no net short).

**User constraints incorporated**:

- Do not buy NO in dead bins (bins with upper bound $U_i <$ current count $n$).
- No pre-trade oracle checks required (assume user validated counting rules already).
- No simulation for exit; use settlement probabilities $p_i$ directly.
- No explicit exit logic: trade only until T_stop; after T_stop, no further trading and hold to settlement.
- Ignore kill switches, child orders, and post-only flags. Orders may still partially fill.

# 1. Definitions and Inputs

**Bins**: There are K bins indexed i=1..K. Bin i corresponds to an integer tweet-count range [L_i, U_i] (inclusive). Exactly one bin is assumed to win at settlement. If not exhaustive, add an extra outcome 'OTHER' - see Section 8.2.

**State**: At time t (t < T_stop) the bot has:

- Current tweet count n = n(t) (integer).
- Time to settlement and cutoff: settlement time T, trading cutoff time T_stop (T_stop < T).
- Model settlement probabilities p_i = P(bin i wins at T | n, t). These are provided by your model.
- Orderbooks for each bin i, separately for YES and NO, with price/size levels for bids and asks.
- Current holdings q_i^Y >= 0 (YES shares) and q_i^N >= 0 (NO shares) and cash C >= 0.
- Hard caps: event collateral cap C_event_max, and per-bin collateral caps C_bin_max[i].
- Fee model: fee_rate_buy, fee_rate_sell (can be equal). Apply to effective execution prices.

**Dead bin (monotone count)**: Bin i is dead at time t if U_i < n. Under monotone count, a dead bin cannot win. Implement by forcing p_i = 0 for any dead bin. (This also implies YES in dead bins has zero expected payoff.)

# 2. Settlement Payoffs and Terminal Wealth

The bot holds positions until settlement. Terminal wealth is computed by scenario (which bin wins).

**Payoffs**:

- YES_i pays 1 if bin i wins, else 0.
- NO_i pays 1 if bin i does NOT win, else 0.

**Terminal wealth by outcome**:

Let outcome j mean 'bin j wins'. With cash C and holdings q^Y, q^N at settlement, terminal wealth in outcome j is:

```
W_j = C + q_j^Y + sum_{i != j} q_i^N
```

Explanation: only YES for the winning bin pays; all NO positions except NO_j pay 1.

**Required boundary condition**: W_j must be strictly positive for all j to use log utility. Enforce W_j >= W_floor where W_floor is a small positive constant (e.g., 1e-6 or $1) at all times before accepting a buy.

# 3. Objective: Expected Log-Utility (Kelly)

The bot sizes positions by maximizing expected log terminal wealth:

```
U = sum_{j=1..K} p_j * log(W_j)
```

This is multi-outcome Kelly. It automatically penalizes portfolios that cause large losses in any plausible outcome, because log(W_j) drops sharply as W_j approaches zero.

**Fractional Kelly (optional but recommended)**: implement either (a) require a positive utility gain greater than a threshold tau > 0 before trading, or (b) scale desired position changes by kappa in (0,1], e.g., 0.25. This reduces sensitivity to model error and illiquidity.

# 4. Exact Marginal-Utility (Trade/No-Trade) Conditions

Because the orderbook is discrete and the portfolio is constrained (no shorts), the bot uses incremental trades. For each candidate trade, it computes the marginal change in U and accepts trades that increase U, subject to caps and boundary conditions.

**Precompute** current scenario wealth and an auxiliary sum:

```
For each outcome j:
  W_j = C + q_j^Y + sum_{i != j} q_i^N
S = sum_{j=1..K} p_j / W_j
```

S must be computed using the current p_j (with dead bins forced to 0) and the current W_j.

## 4.1 Buying YES in bin i

Let c be the effective per-share cost to buy YES_i now (including fees and any VWAP slippage for the requested chunk size). Buying an infinitesimal epsilon shares changes utility with derivative:

```
dU/dε (buy YES_i) = (p_i / W_i) - c * S
```

**Decision rule**: buy YES_i only if (p_i / W_i) > c * S and all constraints remain satisfied after the purchase.

Define the Kelly reservation price for YES_i:

```
c*_YES[i] = (p_i / W_i) / S
```

Then the rule is: buy YES_i if c < c*_YES[i].

## 4.2 Buying NO in bin i

Let c be the effective per-share cost to buy NO_i now. Derivative:

```
dU/dε (buy NO_i) = S*(1 - c) - (p_i / W_i)
```

Define the Kelly reservation price for NO_i:

```
c*_NO[i] = 1 - c*_YES[i]
```

**Decision rule**: buy NO_i only if c < c*_NO[i], the bin is not dead, and all constraints remain satisfied.

**User simplification**: if bin i is dead (U_i < n), do not buy NO_i even if the above condition would suggest it.

## 4.3 Selling positions you already hold

Because you can sell any held positions, selling is simply the reverse test using the effective per-share sale receipt r (bid price net of fees and slippage for a chunk).

- Sell YES_i if (p_i / W_i) - r * S < 0 (equivalently r > c*_YES[i]).
- Sell NO_i if S*(1 - r) - (p_i / W_i) < 0 (equivalently r > c*_NO[i]).

# 5. Effective Execution Prices (Fees and Slippage)

All decisions must use effective prices. In thin books, using only top-of-book prices causes systematic overtrading and losses.

## 5.1 Compute VWAP for a requested chunk size

To buy size s from an ask ladder (price,qty) pairs, consume levels until total qty >= s. The average execution price is:

```
VWAP_ask(s) = (sum_k price_k * filled_qty_k) / s
```

Similarly for selling into bids:

```
VWAP_bid(s) = (sum_k price_k * filled_qty_k) / s
```

If the book depth is insufficient to fill s, mark the candidate as infeasible (or reduce s).

## 5.2 Apply fees

Let fee_buy be the total proportional fee applied to buys, and fee_sell for sells. Use:

```
c_buy = VWAP_ask(s) * (1 + fee_buy)
r_sell = VWAP_bid(s) * (1 - fee_sell)
```

Use c_buy and r_sell in the marginal utility formulas.

Note: If fees are charged in a non-linear way, replace these with the exact fee function; Claude should implement fee as a pluggable function.

# 6. Collateral Caps and Feasibility Checks

Since you plan to hold to settlement, a conservative 'collateral at risk' for long YES/NO is the total dollars paid for open positions (worst-case payoff is 0). Track average entry costs for each instrument.

Maintain per-instrument average entry price (cost basis): avgY[i], avgN[i]. Update on fills using weighted averages.

```
collateral_used_event = sum_i (qY[i] * avgY[i] + qN[i] * avgN[i])
collateral_used_bin[i] = qY[i] * avgY[i] + qN[i] * avgN[i]
```

Hard constraints:

```
collateral_used_event <= C_event_max
collateral_used_bin[i] <= C_bin_max[i]  for all i
```

When evaluating a candidate buy chunk, compute the incremental collateral increase using the effective buy price for that chunk (or update avg* accordingly) and reject if it would breach caps.

# 7. Complete Trading Algorithm (Greedy Constrained Kelly)

Because the CLOB is discrete and illiquid, implement a greedy incremental optimizer that repeatedly selects the trade chunk with the largest positive utility improvement until no further improvements exist or caps bind.

## 7.1 Parameters to specify

• delta: chunk size in shares (e.g., 10 or 50). Smaller delta approximates continuous Kelly more closely.
• tau: optional minimum utility slope/gain threshold to trade (fractional Kelly via edge threshold).
• T_stop: trading cutoff time. After T_stop, no new orders and no rebalancing; hold to settlement.
• W_floor: minimum allowed W_j after any buy (strictly positive).
• max_iters_per_tick: prevent infinite loops; e.g., 100 chunks per tick.

## 7.2 Tick loop (runs while t < T_stop)

```
Inputs each tick:
  n(t), time t
  model probs p[1..K]
  orderbooks YES/NO for each bin
  current holdings qY,qN and cash C
  avg entry prices avgY,avgN

Step A - apply dead-bin logic:
  For each i:
    if U[i] < n(t): p[i] = 0
  (Optionally renormalize p across remaining outcomes; see Section 8.)

Step B - build candidate trade chunks:
  For each bin i:
    YES buy candidate:
      if p[i] > 0 and sufficient ask depth for delta:
         c = effective_buy_cost_Y(i, delta)  # VWAP asks + fees
         compute utility_slope = (p[i]/W[i]) - c*S
         if utility_slope > 0 and passes caps and W_floor: candidate
    YES sell candidate:
      if qY[i] > 0 and sufficient bid depth for delta:
         r = effective_sell_receipt_Y(i, delta)  # VWAP bids - fees
         compute utility_slope_sell = (p[i]/W[i]) - r*S
         if utility_slope_sell < 0: candidate to SELL

    NO buy candidate:
      if (U[i] >= n(t)) and sufficient ask depth and p[i] not forced dead:
         c = effective_buy_cost_N(i, delta)
         utility_slope = S*(1 - c) - (p[i]/W[i])
         if utility_slope > 0 and passes caps and W_floor: candidate
      # User rule: if bin dead, do NOT create NO buy candidates.

    NO sell candidate:
      if qN[i] > 0 and sufficient bid depth:
         r = effective_sell_receipt_N(i, delta)
         utility_slope_sell = S*(1 - r) - (p[i]/W[i])
         if utility_slope_sell < 0: candidate to SELL

Step C - greedy selection:
  Recompute W[1..K] and S using current holdings.
```

```
    Evaluate all candidates' expected utility gain for the chunk:
        approx_gain = delta * utility_slope   (buy)
        approx_gain = delta * (-utility_slope_sell) (sell)
    Choose candidate with largest approx_gain.
    If largest approx_gain <= tau (or <= 0 if tau not used): stop.
    Otherwise submit ONE order for that chunk and wait for fill/partial fill.
    Update C, qY/qN, avgY/avgN based on actual fill.
    Repeat up to max_iters_per_tick.

  Stop condition:
    No positive-gain candidate, or caps bind, or t >= T_stop.
```

## 7.3 Notes on execution without child orders

Even without explicit child orders, the implementation must still handle partial fills. After every fill event, recompute W_j, S, and rebuild candidates using the updated orderbook snapshot. Do not assume the full delta fills.

If you submit one limit order with price that crosses the spread, the matching engine may fill across multiple price levels. That is why effective_buy_cost_* must be computed as VWAP of the expected fill ladder and the submitted limit should be set to the worst price level you are willing to accept.

# 8. Boundary Conditions and Edge Cases (Must Implement)

These conditions are where most logical bugs occur. Implement explicitly.

## 8.1 Probability sanity

After forcing p[i]=0 for dead bins, ensure probabilities are consistent. Two acceptable options:

Option 1 (recommended if bins are exhaustive): renormalize remaining p so sum(p)=1. This preserves the interpretation that exactly one of the remaining live bins must occur.

Option 2 (if your model already outputs a valid distribution including dead bins as exactly 0): do not renormalize.

Do not accidentally leave sum(p) < 1 with no OTHER outcome, because then expected utility will be mis-scaled and reservation prices become distorted.

## 8.2 Non-exhaustive bins

If the market bins do not cover all possible counts, add an extra outcome 'OTHER'. Let its probability be p_OTHER = 1 - sum_{i=1..K} p_i (after any dead-bin adjustments). Then define wealth in OTHER outcome as:

```
W_OTHER = C + sum_{i=1..K} q_i^N  # because no YES_i pays, and all NO_i pay 1
```

Then include OTHER in the S sum: S = sum_over_outcomes p_outcome / W_outcome. This keeps the Kelly math correct.

## 8.3 Dead bins

If U_i < n(t): p_i=0, do not buy YES_i, do not buy NO_i (user rule). You may still sell any YES_i or NO_i you already hold if liquidity exists.

## 8.4 Wealth floor

Before accepting a buy chunk, compute post-trade W'_j for all outcomes and require W'_j >= W_floor. For buy YES_i of cost c*delta:

```
Cash decreases by c*delta:
  C' = C - c*delta
Holdings change:
  qY[i]' = qY[i] + delta
Compute W'_j using the definition in Section 2 and require all W'_j >= W_floor.
```

Analogous for buying NO_i (qN[i] increases). Selling always increases cash so W_floor is not binding for sells.

## 8.5 Tick size and rounding

Polymarket prices are typically in cents. Ensure reservation prices and limit prices are rounded correctly to the nearest tick. When comparing c < c*, treat equality as no-trade unless you explicitly want to trade at zero edge.

## 8.6 Do not hold both YES and NO in the same bin (optional simplification)

Optional: to simplify and reduce self-cancellation, enforce at most one side held per bin: if qY[i] > 0, do not buy NO_i unless you first sell down qY[i] to 0 (and vice versa). This is not required by Kelly math but often simplifies behavior in thin books.

# 9. Implementation Checklist (What Claude Must Produce)

• Data structures for bins: arrays L[i], U[i], and current n(t).
• Functions to compute terminal wealth vector W for all outcomes given C, qY, qN.
• Functions to compute S = sum p/W including optional OTHER outcome.
• Functions to compute VWAP_ask and VWAP_bid for a requested size from an orderbook ladder.
• Fee model function fee_buy(price, size) and fee_sell(price, size) or simple multiplicative rates.
• Candidate generation for YES/NO buy/sell chunks with feasibility checks: depth, caps, W_floor, dead-bin rules.
• Greedy loop that selects the candidate with maximum positive approximate utility gain per iteration, submits one order, processes fills, updates cash/holdings/cost basis, and repeats.
• Cutoff logic: stop trading at T_stop and hold to settlement.
• Unit tests for boundary conditions: dead bins; p normalization; OTHER outcome; W_floor enforcement; partial fills; insufficient depth.

End of spec.